Universidade Federal de Minas Gerais
Colégio Técnico - COLTEC
Prof. Márcio Fantini
Cursos: Automação, Eletrônica e Desenv. de Sistemas
Aula Prática: Estudando Memória e Endereçamento
Parte 1: Arquivos

Nome: _____Turma/Sub:_____ Data: _____

Objetivos Principais do Estudo

(1) Rever conceitos básicos de programação C para leitura e escrita em arquivos; (2) rever conceitos básicos dos comandos do terminal do Ubuntu; (3) usar comandos do shell para verificar conteúdo de arquivos; (4) usar comandos do shell para verificar endereços e conteúdos da RAM; (5) Estudar programas em C que lidam com Hardware

Este documento é um **estudo orientado:** acompanhe as explicações (se precisar, assista as videoaulas) e faça os exercícios pedidos. Estude com o material da apostila.

1 Arquivos Textos e Binários

Tarefa 1

Faça um programa em C para salvar dois arquivos: um binário e um texto. Em ambos salve as variáveis x, y e z com os valores dados a seguir:

```
variável x uma string: "ABCDEF55";
variável y um inteiro: 0xABCDEF55;
variável z uma string: "coltec"
```

Tarefa 2

No terminal do Ubuntu use o comando **hd** para ver o conteúdo dos dois arquivos em bytes.

Entenda a saída do comando hd.

Escreva, no quadro abaixo o conteúdo (em bytes) desses arquivos, como mostrado no terminal. Escreva apenas as variáveis x e y, como mostrado pelo comando **hd**.

```
Conteúdo Arquivo Texto:


Conteúdo Arquivo Binário:
```

Tarefa 3

Abra os arquivos texto e binário em um editor de texto. Se for abrir pelo terminal, pode usar o editor **nano** ou os editores **vi** ou **vim**. Se for abrir no modo de janela, pode usar o **gedit** ou **sublime**. Ou use outro editor de sua preferência ou atê mesmo alguma IDE que já está acostumado a trabalhar.

1. Compare os resultados do que vê quando abre o arquivo texto no editor de texto do que vê quando usa o programa **hd**. É o mesmo conteúdo? (os dois mostram exatamente o mesmo conteúdo?).
2. Faça o mesmo para o arquivo binário.

Explique:

```
Arquivo Texto:

Arquivo Binário:
```

Tarefa 4

Responda as questões pedidas.

1. Qual a diferença entre arquivo binário e texto?
2. Nas tarefas 1 e 2 trabalhamos com dois arquivos, um binário e um texto.
   (a) Use, no terminal o comando **cat** para ver o conteúdo dos dois arquivos.
   (b) Compare o que vê na saída do comando **cat** com o que vê quando abre os arquivos no editor de texto e quando vê os bytes pelo comando **hd**.
   (c) Informe o tamanho deles, em bytes.
   (d) Os dois arquivos salvaram os mesmos valores. Por que quando mostramos o conteúdo deles, em bytes, eles não apresentaram o mesmo resultado? (ou seja, o conteúdo é diferente quando "olhamos" os bytes?)
   (e) Em relação ao tamanho dos arquivos, o que os torna diferentes?

Tarefa 5

Faça um programa em C para definir uma estrutura (struct) contendo três variáveis: duas strings (de trÊs caracteres cada uma) e um inteiro. Coloque valores para as strings e defina $dois = 0x01234567$. Salve a estrutura em um arquivo binário. Veja o conteúdo do arquivo binário usando os comandos **hd** e **cat**. Entenda o resultado.

Avaliação

Você achou essa parte 1 do roteiro:

- fácil ☐
- médio ☐
- difícil ☐
- não sei dizer ☐

Universidade Federal de Minas Gerais
Colégio Técnico - COLTEC
Prof. Márcio Fantini
Cursos: Automação, Eletrônica e Desenv. de Sistemas
Estudando Endereçamento **Parte 2: Arquivos e Numeração**

Nome: _____Turma/Sub:_____ Data: _____

1 Arquivos Textos e Binários

Tarefa 6

Estude as seções 1, 2 e 6 do arquivo BitsBytesArmazenamentoNumeracao_2025.pdf disponibilizado no classroom. Faça os exercícios pedidos nessas seções.

Tarefa 7

Rode novamente o programa da tarefa 1 e anote abaixo os valores das variáveis x e y, em bytes, gravadas no arquivo binário criado. Diga se os dados salvos estão no formato Little Endian ou Big Endian.

```
Valor em Bytes da variável x:
Valor em Bytes da variável y:
```

Tarefa 8

Repita a tarefa anterior, agora para o arquivo texto criado.

```
Valor em Bytes da variável x:
Valor em Bytes da variável y:
```

Tarefa 9

1. Converta o número 12345 dado na base 10 para a base 16.
2. Converta o número 12345 dado na base 10 para a base 2.
3. Converta o número 0x4640E400 dado na base 16 para a base 2.
4. Faça um programa em C para salvar dois arquivos: um binário e um texto. Em ambos salve as variáveis x e y com os valores: **int x = 12345**; **float y = 12345**; Ao salvar a variável inteira no arquivo texto, salve-a em hexadecimal.
5. Use o comando **cat** para ver o conteúdo dos arquivos gerados. Escreva-os no quadro abaixo:

   ```
   Conteúdo Arquivo Texto:
   Conteúdo Arquivo Binário:
   ```

6. Explique o que foi mostrado, as diferenças entre eles e a(s) causa(s) para essa(s) diferença(s).

7. Abra os dois arquivos gerados em um editor de texto (no terminal ou como janela). Entenda as diferenças entre os valores dados nos arquivos. Compare-os com os valores das conversões para a base 16.

Tarefa 10

Altere o programa da tarefa anterior para:

1. salvar a variável inteira no arquivo texto em dois formatos: base 10 e base 16;
2. e salvar no arquivo binário a variável inteira, a variável float usando **sizeof(float)** e a variável float usando **sizeof(double)**.

Visualize o conteúdo dos arquivos com o comando **hd** e anote no quadro a saída dos arquivos, destacando qual foi o valor da saída para cada variável. Apresente os resultados na base 16 (como aparecem quando usamos o comando **hd**.

```
Conteúdo Arquivo Texto - variável x (gravada na base 10):
Conteúdo Arquivo Texto - variável x (gravada na base 16):
Conteúdo Arquivo Texto - variável y (gravada na base 16):
Conteúdo Arquivo Binário - variável x:
Conteúdo Arquivo Binário - variável y (sizeof float):
Conteúdo Arquivo Binário - variável y (sizeof double):
```

Tarefa 11

Escreva os valores do quadro anterior (escritos em hexadecimal) em binário (ou seja, o que escreveu no quadro anterior em hexadecimal mude para a base 2):

```
Conteúdo Arquivo Texto - variável x (gravada na base 10):

Conteúdo Arquivo Texto - variável x (gravada na base 16):

Conteúdo Arquivo Texto - variável y (gravada na base 16):

Conteúdo Arquivo Binário - variável x:

Conteúdo Arquivo Binário - variável y (sizeof float):

Conteúdo Arquivo Binário - variável y (sizeof double):
```

Avaliação

Você achou essa parte do roteiro:

- fácil ☐
- médio ☐
- difícil ☐
- não sei dizer ☐

Compilado 6 de março de 2025 - 10:05:58- fantini@ufmg.br

Universidade Federal de Minas Gerais
Colégio Técnico - COLTEC
Prof. Márcio Fantini
Cursos: Automação, Eletrônica e Desenv. de Sistemas Estudando Endereçamento **Parte 3: Numeração e Memória RAM**

PARTE 3

Nome: _____Turma/Sub:_____ Data: _____

1 Numeração

Tarefa 12

Nas tarefas 9 a 11 trabalhamos com o número 12345 (dado na base 10) gravado em arquivo texto e binário, como inteiro e como float. Esse número quando dado na base 16 é $0x3039$, ou $0x00003039$, quando usamos 4 bytes para representá-lo.

Vimos que esse número, quando armazenado como inteiro foi salvo como: $0x39300000$, indicando que nosso sistema usa a notação Little End.

Vimos que o número, quando armazenado como float é salvo como $0x00E44046$, logo sabemos que o número vale $0x4640E400$

Pede-se:

```
1) Escreva o número 0x3039 na base 2 usando 32 bits:

2)Escreva o número 0x34640E420 na base 2 usando 32 bits:

3) Verifique se há alguma semelhança dos bits das duas representações. Caso haja, escreva aqui:
```

Tarefa 13 - Opcional

Essa tarefa é opcional...por enquanto...).

Pesquise sobre a representação de números float e como o C os representa. Pesquise sobre o padrão IEEE para representação de números float.

2 Memória RAM

Até agora vimos como dados são armazenados em arquivos. Vimos que são salvos em binário e que podemos ver os valores usando comandos e editores do Linux. Vimos sobre os formatos Little Endian e Big Endian.

Importante lembrar que a estrutura de armazenamento em disco é diferente da estrutura do armazenamento na RAM. Apesar de usarmos os mesmos conceitos de organização em bytes, representada pela base 16, a forma que são estruturadas eletronicamente é bem diferente.

Vamos agora estudar um pouco sobre o armazenamento na RAM.

Tarefa 14

Faça um programa em C que define uma variável char $x =' u'$, uma variável inteira $y = 3$, uma variável float $w = 3.14$ e uma variável double $z = 2.7183$. Mostre os valores e os endereços. Monte no espaço abaixo a pilha da memória RAM, com endereços crescendo de baixo para cima, mostrando os endereços e os conteúdos. (se achar melhor, para economizar espaço, monte a pilha na horizontal, da esquerda para direita).

Tarefa 15

Faça um programa em C que define uma variável array de char $x = \{'a','b'\}$, um array de inteiros $y = [1, 2, 3]$, uma variável float $w = 3.14$ e uma variável double $z = 3.14$. Mostre os valores e os endereços. Monte no espaço abaixo, a pilha da memória RAM, com endereços crescendo de baixo para cima, mostrando os endereços e os conteúdos. Mostre os valores de cada elemento dos arrays. Entenda o resultado. (Se achar melhor, para economizar espaço, monte a pilha na horizontal, da esquerda para direita).

Tarefa 16 (Desafio)

Nos dois programas acima, quando visualizamos os endereços eles mostram apenas o endereço inicial de cada variável. Por exemplo, se o inteiro y estiver no endereço ABC, sabemos que ele ocupa as células da RAM de endereço ABC, ABD, ABE e ABF pois um inteiro ocupa 4 bytes.

Altere o programa da Tarefa 14 para mostrar não apenas os endereços iniciais das variáveis, mas sim todos os endereços de todas as células. Mostre os endereços e os conteúdos de cada célula de todas variáveis do problema.

Avaliação

Você achou essa parte do roteiro:

- fácil ☐
- médio ☐
- difícil ☐
- não sei dizer ☐

Compilado 6 de março de 2025 - 10:03:33- fantini@ufmg.br

Universidade Federal de Minas Gerais
Colégio Técnico - COLTEC
Prof. Márcio Fantini
Cursos: Automação, Eletrônica e Desenv. de Sistemas
Estudando Endereçamento **Parte 4: Memória RAM**

Nome: _____Turma/Sub:_____ Data: _____

1 Memória RAM

Até agora usamos recursos do C, como os operadores & ∗ para vermos endereços e conteúdos da memória RAM.

Vamos agora usar comandos do Shell do Ubuntu (Terminal) para ver conteúdo e endereços na RAM.

Tarefa 17

Faça um programa em C que lê um número inteiro na base 10, mostre o endereço da variável e seu valor em hexadecimal e fique esperando o usuário entrar uma tecla para terminar o programa. É importante que o programa continue rodando enquanto o usuário não teclar algo.

```
1)Escreva aqui o endereço da variável

2)Escreva o conteúdo do endereço da variável, em hexadecimal

3)Escreva o conteúdo do endereço da variável, em hexadecimal, na formatação Little Endian.
```

Para fazer a próxima tarefa mantenha o programa rodando.

1.1 Processos no Linux

No Linux os programas que são executados são chamados de **processos**.

Uma forma de vermos os processos que estão sendo executados no Linux é a partir do comando **ps**. Veja o exemplo a seguir, em que mostro parte do resultado do comando **ps -aux**

```
mf@mdell:~$ ps -aux
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.1  0.0 169060 13524 ?        Ss   mar04   0:46 /sbin/init sp
mysql       1178  0.5  1.2 2374844 400160 ?      Ssl  mar04   2:32 /usr/sbin/mys
postgres    1192  0.0  0.0 221104  9376 ?        Ss   mar04   0:00 postgres: 14/
mf          2448  0.0  0.0 105468  6496 ?        S    mar04   0:00 (sd-pam)
www-data   58126  0.0  0.0 280216 25644 ?        S    00:00   0:00 /usr/sbin/apa
mf         91869  2.6  0.8 1212448248 266908 ?   Sl   03:41   0:57 /opt/brave.co
```

Ao executarmos o comando, a saída será uma lista de todos os processos em execução. Se quisermos procurar por um processo específico, podemos usar o **pipe** do Linux.

O pipe (|) é um operador muito útil na linha de comando do terminal em sistemas Unix (como o Ubuntu). Ele permite que você conecte a saída de um comando à entrada de outro comando, formando uma cadeia de comandos.

O comando **grep** é uma ferramenta de busca usada para procurar padrões de texto dentro de arquivos ou de entradas fornecidas. Podemos usar o comando **grep** em conjunto com **ps -aux** para procurar processos específicos.

Tarefa 18

Com o programa da tarefa anterior rodando, execute, em outra janela do Terminal o comando **ps -aux** com o **pipe** para **grep** procurando o arquivo executável do seu programa (da tarefa anterior, que está sendo executado).

```
Anote aqui o número (PID) do processo:
```

Tarefa 19

Sabendo o endereço da variável x, mantendo o programa rodando e sabendo o número do processo (PID), execute o comando:

```
sudo xxd -s  endereço-da-variável  -l BYTES  /proc/PID/mem
Por exemplo, executei:
sudo xxd -s  0x7fff486bfccc -l 4 /proc/54887/mem
para ver 4 bytes do a partir do endereço 0x7fff486bfccc do PID 54887

Anote aqui o resultado:
```

Compare o resultado anotado com o conteúdo da variável anotado na tarefa 17.

Tarefa 20

Faça um programa em C para definir um vetor de inteiros de três elementos.Mantenha o programa rodando e examine a posição da RAM a partir do endereço inicial do vetor.

Use o comando xxd para verificar o endereço e conteúdo desse vetor na RAM.

```
Anote aqui o resultado:

```

Atenção: para acessar a pasta **/proc/**$< PID >$**/mem/** é preciso ter permissão de root (sudo)

Avaliação

Você achou essa parte do roteiro:

- fácil ☐
- médio ☐
- difícil ☐
- não sei dizer ☐

Compilado 6 de março de 2025 - 10:40:38- fantini@ufmg.br

Universidade Federal de Minas Gerais
Colégio Técnico - COLTEC
Prof. Márcio Fantini
Cursos: Automação, Eletrônica e Desenv. de Sistemas
Aula Prática: Estudando Memória e Endereçamento
Parte 5: Memória RAM e GDB

Nome: _____Turma/Sub:_____ Data: _____

1 Memória RAM

Tarefa 21

Vamos continuar estudando alguns comandos do terminal para examinarmos a memória ram.

No exemplo dado a seguir, criei um arquivo .sh, que roda uma linguagem " Shell Script". Nela temos opçao de entrar na linha de comando os dados que precisamos para ver a memória RAM:

```
#!/bin/bash
# Solicita as entradas do usuário
read -p "Digite o endereço inicial (em hexadecimal, ex: 0x7fff8591354c): " endereco
read -p "Digite o ta manho do bloco a ser lido: " tamanho
read -p "Digite o PID do processo: " processo
# Remove o prefixo '0x' do endereço, se existir
endereco=${endereco#0x}
# Executa o comando xxd
sudo xxd -s $((16#$endereco)) -l $tamanho -c 1 /proc/$processo/mem
```

Para executá-lo, mude a permissão para **+x** e execute fazendo *./nome_do_arquivo.sh*. Para essa tarefa pede-se:

1. Salve o programa acima no seu editor favorito.
2. Rode o programa C da **Tarefa 17**, deixando-o ativo. Anote o endereço da variavel: _____
3. Veja o número do processo. Você pode usar o comando **ps -aux | grep nomeArquivoExecutavel** ou simplesmente fazer **pidof nomeArquivoExecutavel.**. Anote aqui o número do processo: _____
4. Defina quantos bytes deseja ver a partir do endereço da variável.
5. Execute o programa do shell dado acima, informando os dados pedidos (informe quantos bytes deseja ver a partir do endereço da variável). Entenda o resultado.
6. Responda: O valor da variável foi apresentado no formato Little Endian ou Big Endian? _____

Tarefa 22

Faça um programa em C que define (e insere valores) um array de inteiros de 4 posições e um array de char de 5 posições. O programa deve ficar rodando enquanto se analisa as variáveis na memória RAM. Use o Shell Script da tarefa anterior para mostrar os endereçcos (célula por célula) e conteído dos arrays definidos. Anote, no formato de pilha de ram (com endereço e conteúdo), destacando para cada célula qual a posição do vetor ela se refere.

Atenção: para acessar a pasta **/proc/**$< PID >$**/mem/** é preciso ter permissão de root (sudo)

Tarefa 23 - Usando o gdb

O GDB (GNU Debugger) é um programa usado para depurar (ou "debugar", *debug*) programas compilados em C, C++ e Assembly. Ele nos permite executar um programa passo a passo, inspecionar variáveis, e identificar erros durante a execução. Vamos usar o GDB para aprender um pouco mais sobre a memória RAM e para iniciarmos sobre Assembly.

GDB para Programas em C

Para usar o GDB temos que compilar o programa em C com a opção -g, que inclui informações de depuração no programa executável. Faça os passos dados a seguir para iniciar o estudo com o GDB. Vamos usar o programa em C feito na **Tarefa 14**.

1. Compile seu programa em C fazendo:

   ```
   gcc -g  meu_programa.c -o  meu_programa_executavel.exe
   ```

2. Inicie o GDB fazendo:

   ```
   gdb ./meu_programa_executavel.exe
   ```

3. Para executar sem programa dentro do GBD digite *run* ou *r*.

   ```
   (gdb) run
   ```

4. Coloque um breakpoint antes da linha que imprime os valores das variáveis. (Considere essa linha de número n). Para colocar breakpoint na linha $n = 10$, por exemplo, faça

   ```
   (gdb) break 10
   ```

5. Rode seu programa novamente, após coloca o breakpoint. O programa irá parar no breakpoint. Para continuar executando passo a passo use os comandos *next* ou *step*. O comando *next* executa a próxima linha. O comando *step* executa a próxima linha e se ela for uma chamada de função, entra dentro da função.

6. Para ver um variável use o comando *print*:

   ```
   (gdb) print NOME_DA_VARIAVEL
   ```

Para essa tarefa execute os passos dados acima no programa C da Tarefa 14 e entenda os resultados. Vamos trabalhar com GDB mais profundamente nas próximas tarefas.